
NOTE

For additional information, see
<http://www.pygame.org/docs/ref/draw.html>.

The set of primitive calls allows you the flexibility to draw any shapes you wish. Again, the order in which you make calls is important. Think of the order of your calls as layers; elements that are drawn early can be overlaid by later calls to any other drawing primitive function.

Summary

In this chapter I introduced the basics of pygame. You installed pygame on your computer, then learned about the model of event-driven programming and the use of events, which is very different from coding text-based programs. I explained the coordinate system of pixels in a window and the way that colors are represented in code.

To start right at the beginning with pygame, I introduced a 12-section template that does nothing but bring up a window and can be used to build any pygame-based program. Using that framework, we then built sample programs that showed how to draw an image in the window (using `blit()`), how to detect mouse events, and how to handle keyboard input. The next demonstration explained how to build a location-based animation.

Rectangles are highly important in pygame, so I covered how the attributes of a `rect` object can be used. I also provided some example code to show how to play sound effects and background music to enhance the user's enjoyment of your programs. Finally, I introduced how to use pygame methods to draw primitive shapes in a window.

While I have introduced many concepts within pygame, almost everything I showed in this chapter has essentially been procedural. The `rect` object is an example of object-oriented code built directly into pygame. In the next chapter, I'll show how to use OOP in code to use pygame more effectively.